

02477 Bayesian Machine Learning - Exercise 2

```
In [2]: %matplotlib inline
import pylab as plt
import pandas as pd
import jax.numpy as jnp
from jax import random
import seaborn as snb

from scipy.stats import binom as binom_dist

snb.set_style('darkgrid')
snb.set_theme(font_scale=1.)
colors = 'brgmc'
```

Introduction

The purpose of this exercise is to dive deeper into the basic concepts of probabilistic and Bayesian modelling using **logistic regression** as a case-study. Specifically, we will study:

- the probability distributions involved in logistic regression
- how to approximate the posterior distribution of non-conjugate models using simple grid approximations
- how to make predictions using posterior predictive distributions

We will study these concepts using data from Challenger Space Shuttle disaster.

Content

- Part 1: Setting up the logistic regression model
- Part 2: The prior, likelihood and posterior
- Part 3: Making predictions using point estimates
- Part 4: Approximating the posterior using a grid approximation
- Part 5: Propagating uncertainty from parameters to predictions

Dataset: The Challenger Space Shuttle disaster

On a cold morning on January 28, 1986 the space shuttle called **Challenger** tragically exploded 73 seconds after its launch, killing all 7 crew members. The explosion was later determined to be caused the failure of two out of six so-called o-ring seals in one of the rocket boosters. The temperature on the day of the launch was close 0°C , which was record-low. The cold weather made the o-rings stiffen, reducing their ability to seal the critical joints, and hence, causing the tragic accident. O-ring failure had previously been observed on 6 of out the 23 prior launches as plotted below.

In this exercise, you will construct a simple Bayesian logistic regression model for predicting the probability of o-ring failure as a function of temperature.

The dataset $\mathcal{D} = \{(t_i, y_i)\}_{i=1}^M$ consists of $M = 23$ observations, where $y_i \in [0, 1, \dots, 6]$ denotes the number of failed o-rings for the i 'th temperature t_i . The number of o-rings for all launches were $N = 6$, and hence, the fraction of failed o-rings for the i 'th launch is $\frac{y_i}{N} = \frac{y_i}{6}$.

See [Wikipedia](#) for more background information.

First, we will load the data:

```
In [3]: # load data from csv file using pandas
data = pd.read_csv('./o-ring.csv')
t = data['temperature'].values
y = data['failed'].values
N = data['count'].values

data.head()
```

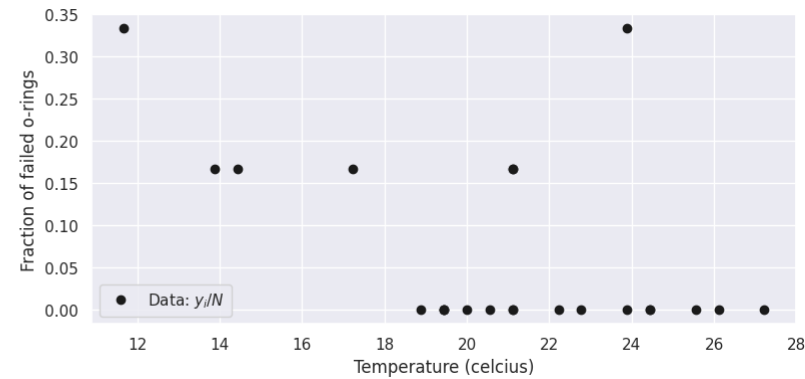
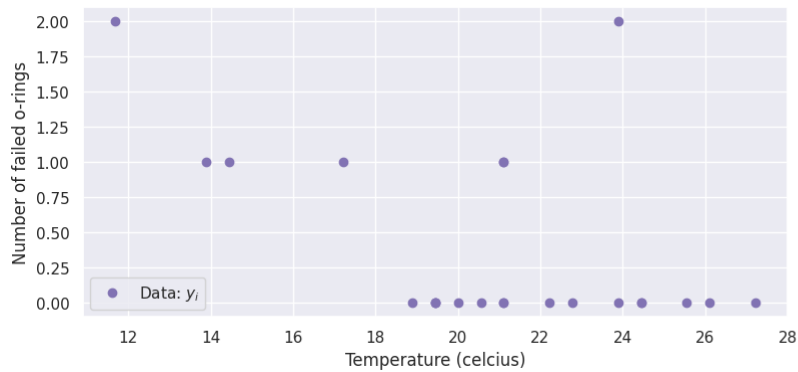
```
Out[3]:
```

	temperature	failed	count
0	18.888889	0	6
1	21.111111	1	6
2	20.555556	0	6
3	20.000000	0	6
4	19.444444	0	6

.. and plot it:

```
In [4]: # helper function for plotting data
def plot_data(ax=None, counts=True):
    if counts:
        ax.plot(t, y, 'mo', label='Data: $y_i$')
        ax.set(ylabel='Number of failed o-rings')
    else:
        ax.plot(t, y/N, 'ko', label='Data: $y_i/N$')
        ax.set(ylabel='Fraction of failed o-rings')
    ax.set(xlabel='Temperature (celcius)')
    ax.legend(loc='lower left')

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 4))
plot_data(ax[0])
plot_data(ax[1], counts=False)
```



Rather than working directly with the temperatures $t_i \in \mathbb{R}$, we will standardize the temperatures to get

$$x_i = \frac{t_i - t_{\text{mean}}}{t_{\text{std}}},$$

where t_{mean} and t_{std} are the empirical mean and standard deviation, respectively, of the observed temperatures $\{t_i\}_{i=1}^M$.

```
In [5]: # compute mean and standard deviation of temperatures
tmean, tstd = jnp.mean(t), jnp.std(t)

# standardize temperatures
standardize = lambda t_: (t_ - tmean)/tstd
x = standardize(t)
```

Part 1: Setting up the logistic regression model

The big picture

Our goal is to use the training data, i.e. the pairs of $\mathbf{x}_i$ and y_i , to learn a model such that we can make predictions and reason about the number of failures y^* for a new input point x^* . In probabilistic terms, our goal is to compute **posterior predictive distribution**

$$p(y^* | \mathbf{y}, \mathbf{x}, x^*).$$

That is, the distribution of the number of failures y^* **conditioned on** the observed data $(\mathbf{x}, \mathbf{y})$ as well as the new input point x^* . Once we computed this distribution, we can use it to reason about y^* and answer questions like:

- What is the expected number of failures $\mathbb{E}[y^* | y, x, x^*]$?
- What is the most likely number of failures $\mathbb{P}(y^* | y, x, x^*)$?
- What is the probability that the number of failures is zero i.e. $\mathbb{P}(y^* = 0 | y, x, x^*)$?
- What is a 90% interval credibility interval for the number of failures?

Before we can compute $p(y^* | \mathbf{y}, \mathbf{x}, x^*)$, we first need to specify a model.

Specifying the model

For each temperature x_i , we know the number of failed o-ring $y_i = \{0, 1, \dots, 6\}$ out of the total number of o-rings $N_i = 6$. Hence, the number of failing o-rings y_i for a given temperature x_i can be described by a **Binomial distribution** with $N_i = 6$ and probability $\theta_i \in [0, 1]$:

$$y_i | \theta_i \sim \text{Bin}(N_i, \theta_i). \quad (1)$$

🔴 **Interpretation:** this is equivalent to modeling the six rings as conditionally independent Bernoulli trials with common probability θ_i for a given x_i .

This is equivalent to modelling the failure of each individual o-ring as **conditionally independent** Bernoulli trials. Since all observations have $N_i = 6$, we drop the subscript i and simply write $N = 6$ in the following to ease the notation.

Rather than imposing Beta-distributions as prior distributions for each θ_i , we assume θ_i can be modelled as is a function of the temperature. Specifically, we assume

$$\theta(x) = \sigma(\alpha + \beta x), \quad (2)$$

where $\alpha, \beta \in \mathbb{R}$ are parameters to be estimated and $\sigma : \mathbb{R} \rightarrow [0, 1]$ is the **logistic sigmoid function**, i.e. $\sigma(x) = (1 + \exp(-x))^{-1}$. For later references, we will denote the linear component of the model as $f(x)$, i.e.

$$f(x) = \alpha + \beta x. \quad (3)$$

Equivalently, the **log-odds** (logits) are linear in temperature:

$$\log\left(\frac{\theta(x)}{1 - \theta(x)}\right) = f(x) = \alpha + \beta x.$$

Note that both $\theta(x)$ and $f(x)$ do indeed depend on α and β , but we suppress this dependency to ease the notation. The likelihood for a single (i 'th) data point thus becomes

$$p(y_i | x_i, \alpha, \beta) = \text{Bin}(y_i | N_i, \theta_i), \quad \text{where} \quad \theta_i = \theta(x_i) = \sigma(\alpha + \beta x_i) = \frac{1}{1 + \exp(-(\alpha + \beta x_i))}. \quad (4)$$

Assuming that all observations $\{y_i\}_{i=1}^M$ are **conditionally independent** given θ_i , the likelihood of the full dataset becomes

$$p(\mathbf{y} | \mathbf{x}, \alpha, \beta) = \prod_{i=1}^M p(y_i | x_i, \alpha, \beta) = \prod_{i=1}^M \text{Bin}(y_i | N, \theta_i), \quad (5)$$

where $\mathbf{y} = [y_1, y_2, \dots, y_M]$ and $\mathbf{x} = [x_1, x_2, \dots, x_M]$. Our goal is to be able to predict y^* for a new temperature x^* . To do this we write down the *predictive likelihood* for y^* , i.e.

$$p(y^* | x^*, \alpha, \beta) = \text{Bin}(y^* | N^*, \theta^*), \quad \text{where} \quad \theta^* = \theta(x^*) = \sigma(\alpha + \beta x^*) = \frac{1}{1 + \exp(-(\alpha + \beta x^*))}. \quad (6)$$

Using the **product rule** and the assumption of conditional independence, we have

$$p(\mathbf{y}, y^* | \mathbf{x}, x^*, \alpha, \beta) = p(\mathbf{y} | \mathbf{x}, \alpha, \beta) p(y^* | x^*, \alpha, \beta) = \underbrace{\prod_{i=1}^M \text{Bin}(y_i | N, \theta_i)}_{p(\mathbf{y} | \mathbf{x}, \alpha, \beta)} \underbrace{\text{Bin}(y^* | N^*, \theta^*)}_{p(y^* | x^*, \alpha, \beta)}. \quad (7)$$

To complete the Bayesian model, we need to impose a prior distribution on the parameters α, β . Since we do not have any information about the parameters, we will impose generic Gaussian priors with zero mean:

$$p(\alpha, \beta) = \mathcal{N}(\alpha | 0, \sigma_\alpha^2) \mathcal{N}(\beta | 0, \sigma_\beta^2) \quad (8)$$

for $\sigma_\alpha^2, \sigma_\beta^2 > 0$. This leads to the following **joint distribution** for $y^*, \mathbf{y}, \alpha$, and β :

$$p(\mathbf{y}, y^*, \alpha, \beta | \mathbf{x}, x^*) = p(y^* | x^*, \alpha, \beta) p(\mathbf{y} | \mathbf{x}, \alpha, \beta) p(\alpha, \beta) = \underbrace{\text{Bin}(y^* | N^*, \theta^*)}_{p(y^* | x^*, \alpha, \beta)} \underbrace{\prod_{i=1}^M \text{Bin}(y_i | N, \theta_i)}_{p(\mathbf{y} | \mathbf{x}, \alpha, \beta)} \underbrace{\mathcal{N}(\alpha | 0, \sigma_\alpha^2) \mathcal{N}(\beta | 0, \sigma_\beta^2)}_{p(\alpha, \beta)}, \quad (9)$$

where we again used the product rule.

🔴 **Why the joint matter:** Why the joint matters: it encodes all assumptions and dependencies. Any other quantity we care about is obtained by conditioning and/or marginalizing this joint.

The joint distribution is a central object because it completely determines how all variables (e.g. both data and parametes) interact and thus, it completely specifies our probabilistic model. From the joint distribution, we can derive any related distribution of interest using the rules of probability theory.

From the joint distribution to the posterior predictive distribution

Because we observe $\mathbf{y}$, we naturally consider the **conditional distribution** of (y^*, α, β) given the observed quantities $(\mathbf{y}, \mathbf{x}, x^*)$. Using the standard rules for conditional distributions, we get:

$$p(y^*, \alpha, \beta | \mathbf{y}, \mathbf{x}, x^*) = \frac{p(\mathbf{y}, y^*, \alpha, \beta | \mathbf{x}, x^*)}{p(\mathbf{y} | \mathbf{x})}. \quad (10)$$

Our primary goal is to compute the **posterior predictive distribution** $p(y^* | \mathbf{y}, \mathbf{x}, x^*)$, i.e. the **marginal distribution** of y^* conditioned on the observed data, i.e. $p(y^* | \mathbf{y}, \mathbf{x}, x^*)$ which can be obtained by marginalizing over α and β via the **sum rule**:

$$p(y^*|\mathbf{y}, \mathbf{x}, x^*) = \int \int p(y^*, \alpha, \beta|\mathbf{y}, \mathbf{x}, x^*)d\alpha d\beta = \int \int \underbrace{p(y^*|x^*, \alpha, \beta)}_{\text{likelihood for } y^*} \underbrace{p(\alpha, \beta)}_{\text{posterior distribution}} d\alpha d\beta,$$

(11)

as we derived in the lecture. The last equation shows that we can obtain the posterior predictive distribution by averaging the predictive likelihood wrt. to the posterior distribution. Hence, our first effort will be to get a handle on the posterior distribution of α and β conditioned on $\mathbf{y}$.

For reference, the posterior over parameters is

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta) p(\alpha, \beta) = \left[\prod_{i=1}^M \text{Bin}(y_i \mid N, \theta_i) \right] \mathcal{N}(\alpha \mid 0, \sigma_\alpha^2) \mathcal{N}(\beta \mid 0, \sigma_\beta^2),$$

with proportionality constant $p(\mathbf{y} \mid \mathbf{x})^{-1}$.

(In logistic regression, this posterior has no closed form, so we typically approximate it via Laplace approximation, variational inference, or MCMC. The predictive integral can then be evaluated by Monte Carlo averaging.)

Quick notation reminders (to reduce cognitive load while reading equations):

- $N = N^* = 6$ throughout (six O-rings per case).
- $\theta_i = \sigma(\alpha + \beta x_i)$, and $\theta^* = \sigma(\alpha + \beta x^*)$.
- $f(x) = \alpha + \beta x$ is the linear predictor.
- $\theta(x) = \sigma(f(x))$.
- Bold symbols $\mathbf{x}, \mathbf{y}$ denote vectors over the M observed cases.

Before starting the practical implementation, we will practice the terminology and the probabilistic machinery.

In []:

Task 1.1: Use the **product rule** to write down the joint distribution of $\mathbf{y}$, α , and β given $\mathbf{x}$:

Hint: This is meant to be an easy task.

Solution

The product rule yields:

$$p(\mathbf{y}, \alpha, \beta|\mathbf{x}) = p(\mathbf{y}|\alpha, \beta, \mathbf{x})p(\alpha, \beta)$$

End of solution

Task 1.2: Show that $p(\mathbf{y}, \alpha, \beta|\mathbf{x}) = \sum_{y^*} p(\mathbf{y}, y^*, \alpha, \beta|\mathbf{x}, x^*)$, where the sum is over all possible outcomes of y^* .

Solution

We have

$$\begin{aligned} \sum_{y^*} p(\mathbf{y}, y^*, \alpha, \beta|\mathbf{x}, x^*) &= \sum_{y^*} p(y^*|x^*, \alpha, \beta)p(\mathbf{y}|\mathbf{x}, \alpha, \beta)p(\alpha, \beta) && \text{(Using eq. (9))} \\ &= p(\mathbf{y}|\mathbf{x}, \alpha, \beta)p(\alpha, \beta) \sum_{y^*} p(y^*|x^*, \alpha, \beta) && \text{(linearity of sums)} \\ &= p(\mathbf{y}|\mathbf{x}, \alpha, \beta)p(\alpha, \beta) && \text{(PMFs sums to 1)} \\ &= p(\mathbf{y}, \alpha, \beta|\mathbf{x}), && \text{(product rule)} \end{aligned}$$

since $\sum_{y^*} p(y^*|x^*, \alpha, \beta) = 1$.

End of solution

Task 1.3: Use Bayes' rule to express the posterior $p(\alpha, \beta|\mathbf{y}, \mathbf{x})$ in terms of the prior, likelihood and evidence and argue that the posterior is proportional to the joint distribution $p(\mathbf{y}, \alpha, \beta|\mathbf{x})$.

Solution

Bayes' rules states that

$$p(\alpha, \beta|\mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y}|\alpha, \beta, \mathbf{x})p(\alpha, \beta)}{p(\mathbf{y}|\mathbf{x})}$$

Furthermore, the **evidence** $p(\mathbf{y}|\mathbf{x}) = k$ is constant wrt. both α and β , and therefore, we get

$$p(\alpha, \beta|\mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y}|\alpha, \beta, \mathbf{x})p(\alpha, \beta)}{k} \propto p(\mathbf{y}|\alpha, \beta, \mathbf{x})p(\alpha, \beta) = p(\mathbf{y}, \alpha, \beta|\mathbf{x})$$

End of solution

Part 2: The prior, likelihood and posterior

The goal of this part is to implement code for evaluating the prior, likelihood and posterior distribution. Due to a combination of **numerical precision** and **mathematical convenience**, we will implement functions for evaluating the logarithm of these distributions. Below you are given an incomplete implementation of the logistic regression model. Your task is to complete the implementation. The code contains a few sanity checks that will help to detect potential bugs and errors in your code.

Task 2.1: Complete the implementation of the functions f , θ , $\log_prior$ and $\log_likelihood$ below.

Hints:

- For implementing the log prior, the function `log_npdf` is handy.
- When implementing the log likelihood, you can either use `*scipy.stats.binom` (imported as `binom_dist`) or you can implement the PMF of the binomial distribution from scratch.*
- Note that for each function the output must have the same shape as the inputs. That is, if you run `log_prior(1, 1)`, it should return a scalar. If you run `log_prior(alpha_vec, beta_vec)`, where `alpha_vec` and `beta_vec` are $[10 \times 1]$ arrays, then the results should also be a $[10 \times 1]$ array

```
In [1]: import jax.numpy as jnp
from jax.scipy.stats import binom # Note: you'll need this import for binom_dist.logpmf

# =====
# UTILITY FUNCTIONS
# =====

# Sigmoid activation function:  $\sigma(x) = 1/(1 + \exp(-x))$ 
# Maps any real number to  $(0,1)$ , used to convert linear outputs to probabilities
# Input shape: (...) -> Output shape: (...)
sigmoid = lambda x: 1./(1 + jnp.exp(-x))

# Log probability density function for normal distribution
#  $\log N(x; \mu, \sigma^2) = -(x-\mu)^2/(2\sigma^2) - 0.5 \log(2\pi\sigma^2)$ 
# Input shapes: x(...), m(...), v(...) -> Output shape: (...)
log_npdf = lambda x, m, v: -(x-m)**2/(2*v) - 0.5*jnp.log(2*jnp.pi*v)

# =====
# BAYESIAN LOGISTIC REGRESSION MODEL
# =====

class LogisticRegression(object):
    """
    Bayesian Logistic Regression for binomial data.

    Model assumptions:
    - Linear relationship:  $f(x) = \alpha + \beta x$  (eq. 3)
    - Sigmoid transformation:  $\theta(x) = \sigma(f(x))$  (eq. 2)
    - Binomial likelihood:  $y \sim \text{Binomial}(N, \theta(x))$  (eq. 5)
    - Normal priors:  $\alpha \sim N(0, \sigma^2_\alpha)$ ,  $\beta \sim N(0, \sigma^2_\beta)$  (eq. 8)

    Use case: Binary classification or proportion modeling where you want
    uncertainty quantification over parameters  $\alpha$  and  $\beta$ .
    """

    def __init__(self, x, y, N, sigma2_alpha=1., sigma2_beta=1.):
        """
        Initialize the logistic regression model.

        Args:
            x: Input features, shape (n_samples,)
            y: Observed successes (counts), shape (n_samples,)
            N: Number of trials for each observation, shape (n_samples,) or scalar
            sigma2_alpha: Prior variance for intercept  $\alpha$ , scalar
            sigma2_beta: Prior variance for slope  $\beta$ , scalar

        Example:
            x = jnp.array([1.0, 2.0, 3.0]) # 3 data points
            y = jnp.array([2, 5, 8]) # successes observed
            N = jnp.array([10, 10, 10]) # trials per data point
        """
        # Data storage
        self.x = x # Features: shape (n_samples,)
        self.y = y # Success counts: shape (n_samples,)
        self.N = N # Trial counts: shape (n_samples,) or scalar

        # Hyperparameters for Bayesian priors
        self.sigma2_alpha = sigma2_alpha # Prior variance for intercept  $\alpha$ 
        self.sigma2_beta = sigma2_beta # Prior variance for slope  $\beta$ 

    def f(self, x, alpha, beta):
        """
        Linear function:  $f(x) = \alpha + \beta x$  (eq. 3)
```

```

This is the linear predictor before sigmoid transformation.

Args:
    x: Input features, shape (... , n_features)
    alpha: Intercept parameter, shape (... , 1) or broadcastable
    beta: Slope parameter, shape (... , 1) or broadcastable

Returns:
    Linear predictions, same shape as x

Note: Output must have the same shape as x for vectorized operations
"""
return alpha + beta * x

def theta(self, x, alpha, beta):
    """
    Success probability:  $\theta(x) = \sigma(f(x)) = \sigma(\alpha + \beta x)$  (eq. 2)

    Applies sigmoid to convert linear predictions to probabilities  $\in (0,1)$ .

    Args:
        x: Input features, shape (... , n_features)
        alpha: Intercept parameter, shape (... , 1) or broadcastable
        beta: Slope parameter, shape (... , 1) or broadcastable

    Returns:
        Success probabilities, same shape as x, values  $\in (0,1)$ 
    """
    return sigmoid(self.f(x, alpha, beta))

def log_prior(self, alpha, beta):
    """
    Log prior probability:  $\log p(\alpha, \beta)$  (eq. 8)

    Assumes independent normal priors:
    -  $\alpha \sim N(0, \sigma^2_\alpha)$ 
    -  $\beta \sim N(0, \sigma^2_\beta)$ 
    -  $\log p(\alpha, \beta) = \log N(\alpha; 0, \sigma^2_\alpha) + \log N(\beta; 0, \sigma^2_\beta)$ 

    Args:
        alpha: Intercept parameter, shape (... , 1)
        beta: Slope parameter, shape (... , 1)

    Returns:
        Log prior probability, shape (... , 1)

    Use case: Regularization - penalizes large parameter values
    """
    log_p_alpha = log_ndpdf(alpha, 0, self.sigma2_alpha)
    log_p_beta = log_ndpdf(beta, 0, self.sigma2_beta)
    return log_p_alpha + log_p_beta

def log_likelihood(self, alpha, beta):
    """
    Log likelihood:  $\log p(y \mid x, \alpha, \beta, N)$  (eq. 5)

    Assumes binomial likelihood for each data point:
     $y_i \sim \text{Binomial}(N_i, \theta(x_i))$  where  $\theta(x_i) = \sigma(\alpha + \beta x_i)$ 

    Total log likelihood =  $\sum_i \log \text{Binomial}(y_i; N_i, \theta(x_i))$ 

    Args:
        alpha: Intercept parameter, shape (... , 1)
        beta: Slope parameter, shape (... , 1)

    Returns:
        Log likelihood, shape (... , 1)

    Note: Uses vectorized operations across all data points
    """
    # Compute success probabilities for all data points
    theta = self.theta(self.x, alpha, beta) # shape: (... , n_samples)

    # Compute log probability for each binomial observation
    # binom.logpmf(k, n, p) = log P(X = k | X ~ Binomial(n, p))
    log_lik = jnp.sum(
        binom.logpmf(self.y, n=self.N, p=theta),
        axis=-1,
        keepdims=True
    )
    return log_lik

```

```

def log_joint(self, alpha, beta):
    """
    Log joint probability:  $\log p(\alpha, \beta, y \mid x, N)$ 

    By Bayes' theorem:
     $\log p(\alpha, \beta \mid y, x, N) \propto \log p(y \mid x, \alpha, \beta, N) + \log p(\alpha, \beta)$ 

    Args:
        alpha: Intercept parameter, shape (..., 1)
        beta: Slope parameter, shape (..., 1)

    Returns:
        Log joint probability, shape (...,)

    Use case:
    - Optimization target for MAP estimation
    - Unnormalized log posterior for MCMC sampling
    - Model comparison via marginal likelihood
    """
    log_prior = self.log_prior(alpha, beta).squeeze()
    log_likelihood = self.log_likelihood(alpha, beta).squeeze()
    return log_prior + log_likelihood

# =====
# USAGE EXAMPLE
# =====

# Example data setup:
# x = jnp.array([1.0, 2.0, 3.0, 4.0]) # predictor variables
# y = jnp.array([1, 3, 7, 9]) # observed successes
# N = jnp.array([10, 10, 10, 10]) # trials per observation

# Instantiate model with default priors
# model = LogisticRegression(x, y, N)

# The model can now be used for:
# 1. MAP estimation: optimize model.log_joint(alpha, beta)
# 2. MCMC sampling: use log_joint as target density
# 3. Variational inference: approximate posterior over (alpha, beta)
# 4. Prediction: model.theta(x_new, alpha_sample, beta_sample)
# instantiate model
model = LogisticRegression(x, y, N)

# some sanity checks to help verify your implementation and make it compatible with the rest of the exercise
assert jnp.allclose(model.theta(x=2, alpha=2, beta=-0.5), 0.7310585786300049), "The value of output of the theta-function was different than expected. Please check your code."
assert jnp.allclose(model.log_prior(1., 1.), -2.8378770664093453), "The value of the output of the log_prior function was different than expected. Please check your code."
assert jnp.allclose(model.log_likelihood(-1., 2.), -95.18926297085957), "The value of the output of the log_likelihood function was different than expected. Please check your code."

assert model.theta(jnp.linspace(-3, 3, 10), 1, 1).shape == (10,), "The shape of the output of the theta-function was different than expected. Please check your code."
assert model.log_prior(jnp.linspace(-3, 3, 10), jnp.linspace(-3, 3, 10)).shape == (10,), "The shape of the output of the log_prior-function was different than expected. Please check your code."
assert model.log_likelihood(jnp.linspace(-3, 3, 10)[:, None], jnp.linspace(-3, 3, 10)[:, None]).shape == (10, 1), "The shape of the output of the log_likelihood-function was different than expected. Please check your code."

-----
NameError                                Traceback (most recent call last)
Cell In[1], line 190
    169         return log_prior + log_likelihood
    172 # =====
    173 # USAGE EXAMPLE
    174 # =====
    (...)    188 # 4. Prediction: model.theta(x_new, alpha_sample, beta_sample)
    189 # instantiate model
--> 190 model = LogisticRegression(x, y, N)
    192 # some sanity checks to help verify your implementation and make it compatible with the rest of the exercise
    193 assert jnp.allclose(model.theta(x=2, alpha=2, beta=-0.5), 0.7310585786300049), "The value of output of the theta-function was different than expected. Please check your code."

```

NameError: name 'x' is not defined

If the model implementation above is correct, then the code below should produce contour plots of the prior, likelihood and posterior. We also compute and plot the MLE and MAP estimators for the model. Because this is a 2D problem, we can simply obtain the MLE/MAP estimators by maximizing over the set of grid values. However, this becomes infeasible in higher dimensions, where gradient-based optimization is used instead.

```

In [6]: class Grid2D(object):
        """ helper class for evaluating the function func on the grid defined by (alpha, beta)"""

        def __init__(self, alphas, betas, func, name="Grid2D"):
            self.alphas = alphas
            self.betas = betas
            self.grid_size = (len(self.alphas), len(self.betas))
            self.alpha_grid, self.beta_grid = jnp.meshgrid(alphas, betas, indexing='ij')
            self.func = func

```

```

self.name = name

# evaluate function on each grid point
self.values = self.func(self.alpha_grid[:, :, None], self.beta_grid[:, :, None]).squeeze()

def plot_contours(self, ax, color='b', num_contours=10, f=lambda x: x, alpha=1.0, title=None):
    ax.contour(self.alphas, self.betas, f(self.values).T, num_contours, colors=color, alpha=alpha)
    ax.set_xlabel('$\\alpha$', ylabel='$\\beta$')
    ax.set_title(self.name, fontweight='bold')

@property
def argmax(self):
    idx = jnp.argmax(self.values)
    alpha_idx, beta_idx = jnp.unravel_index(idx, self.grid_size)
    return self.alphas[alpha_idx], self.betas[beta_idx]

# define grid points (the only reason for using different numbers of points for alpha and beta is to make debugging easier)
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha)
betas = jnp.linspace(-4, 4, num_beta)

# evaluate log prior, log likelihood and log joint on the (alpha, beta)-grid
model = LogisticRegression(x, y, N, sigma2_alpha=1.0, sigma2_beta=1.0)
log_prior_grid = Grid2D(alphas, betas, model.log_prior, 'Prior')
log_lik_grid = Grid2D(alphas, betas, model.log_likelihood, 'Likelihood')
log_joint_grid = Grid2D(alphas, betas, model.log_joint, 'Posterior')

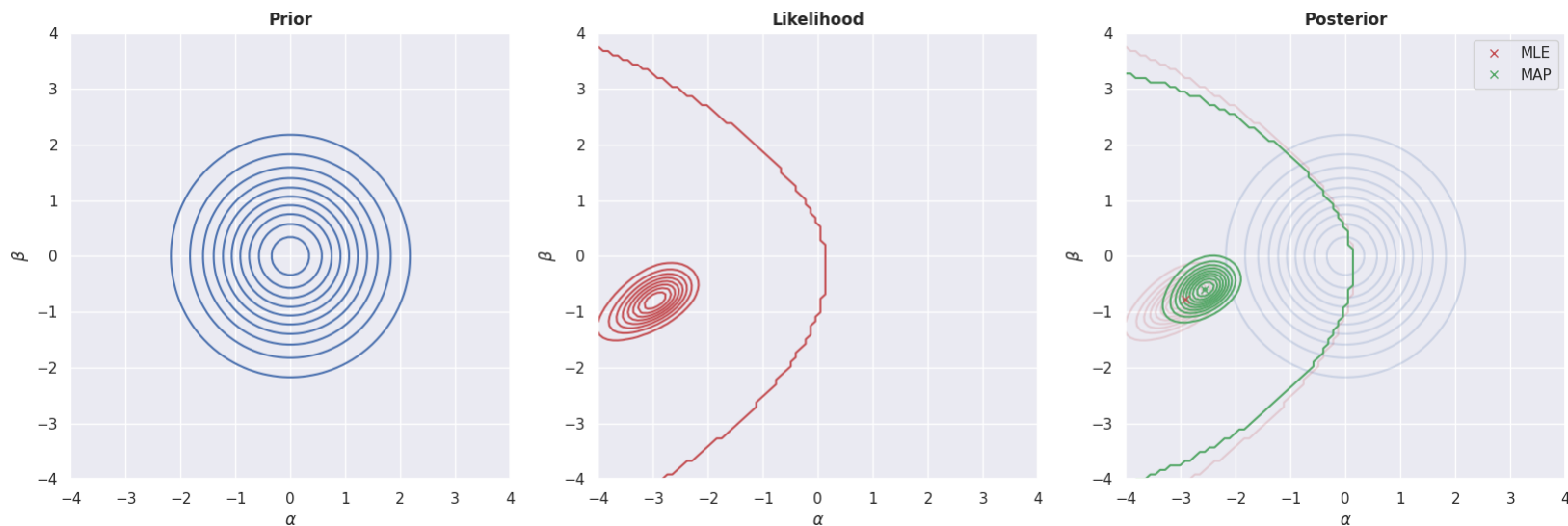
# visualize
fig, ax = plt.subplots(1, 3, figsize=(20, 6))
log_prior_grid.plot_contours(ax[2], f=jnp.exp, color='b', alpha=0.2)
log_lik_grid.plot_contours(ax[2], f=jnp.exp, color='r', alpha=0.2)

for idx_plot, grid in enumerate([log_prior_grid, log_lik_grid, log_joint_grid]):
    grid.plot_contours(ax[idx_plot], f=jnp.exp, color=colors[idx_plot])

# compute and plot MLE and MAP estimators
alpha_MLE, beta_MLE = log_lik_grid.argmax
alpha_MAP, beta_MAP = log_joint_grid.argmax

ax[2].plot(alpha_MLE, beta_MLE, 'rx', label='MLE')
ax[2].plot(alpha_MAP, beta_MAP, 'gx', label='MAP')
ax[2].legend();

```



Task 2.2: When plotting the contours of the posterior distribution, we actually evaluate the joint distribution and not the posterior distribution. Why is this okay when making contour plots?

Solution

Bayes' rule state that the posterior is proportional to the joint distribution. That is, $p(\alpha, \beta | \mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y} | \alpha, \beta, \mathbf{x}) p(\alpha, \beta)}{p(\mathbf{y} | \mathbf{x})} \propto p(\mathbf{y} | \alpha, \beta, \mathbf{x}) p(\alpha, \beta)$ because the evidence $p(\mathbf{y} | \mathbf{x})$ is constant wrt. α and β and hence does not affect the contour plots.

End of solution

Task 2.3: Use the code above to experiment and investigate the relationship between the prior, likelihood and posterior. What happens if you change the prior variances of α, β ? E.g. $\sigma_\alpha^2 = \sigma_\beta^2 = 10$, $\sigma_\alpha^2 = \sigma_\beta^2 = 100$, $\sigma_\alpha^2 = \sigma_\beta^2 = 0.1$? What happens if $\sigma_\alpha^2 = 1$ and $\sigma_\beta^2 = 10$?

Note: This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.

Task 2.4: What happens to the MLE and MAP estimators when you change the variances of the prior?

Note: This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.

Task 2.5: What happens to the prior, likelihood and posterior if you only use a subset of the data? Use the code above to experiment and investigate what happens, e.g. use the first 10 data instead of the full $M = 23$ datapoints.

Note: This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.

Task 2.6: Use the plots to roughly estimate the prior and posterior probability of the event $\beta < 0$.

Solution

The prior distribution is symmetric around $(\alpha, \beta) = (0, 0)$ as seen in the leftmost plot, and hence, the probability of $\beta < 0$ is 0.5:

$$p(\beta < 0) = 0.5.$$

By inspecting the rightmost plot, we see that most of the posterior mass is below the line $\beta = 0$, and hence, we expect the posterior probability of $\beta < 0$ to be close to one:

$$p(\beta < 0 | \mathbf{y}) \approx 1.$$

End of solution

Part 3: Making predictions using point estimates

Our model for the probability of failure for each individual o-ring is given by

$$\theta(x) = \sigma(\alpha + \beta x),$$

where $\alpha, \beta \in \mathbb{R}$ are unknown parameters. Since we do not know the values of the parameters, we can **estimate** them from the data in several ways. The simplest way is via the maximum likelihood estimator (MLE). That is, picking the **point** that maximizes the likelihood of the data:

$$\hat{\alpha}_{\text{MLE}}, \hat{\beta}_{\text{MLE}} = \arg \max_{\alpha, \beta \in \mathbb{R}} p(\mathbf{y} | \mathbf{x}, \alpha, \beta).$$

In practice, we most often due this using gradient-based optimization. However, in this exercise, we will simply pick the parameters values that maximizes the likelihood function on the grid, which works well for 2D problems. Instead of using the MLE, we can also use the Maximum a posteriori (MAP) estimator. That is, picking the **point** that maximizes the posterior distribution:

$$\hat{\alpha}_{\text{MAP}}, \hat{\beta}_{\text{MAP}} = \arg \max_{\alpha, \beta} p(\alpha, \beta | \mathbf{y}, \mathbf{x}).$$

After obtaining an estimate of the parameters, we can use a **plugin approximation** for $\theta(x)$ by simply inserting the **point estimate** (e.g. MLE or MAP) for α and β :

$$\hat{\theta}(x) = \sigma(\hat{\alpha} + \hat{\beta} x),$$

and then use $\hat{\theta}$ for making predictions.

Task 3.1: Compute and plot the predicted probability $\hat{\theta}(x)$ for x corresponding to temperatures $t \in [-10, 40]$ for both MLE and MAP estimators.

```
In [7]: # data for predictions
temp_pred = jnp.linspace(-10, 40, 500)
x_pred = standardize(temp_pred)
N_pred = 6

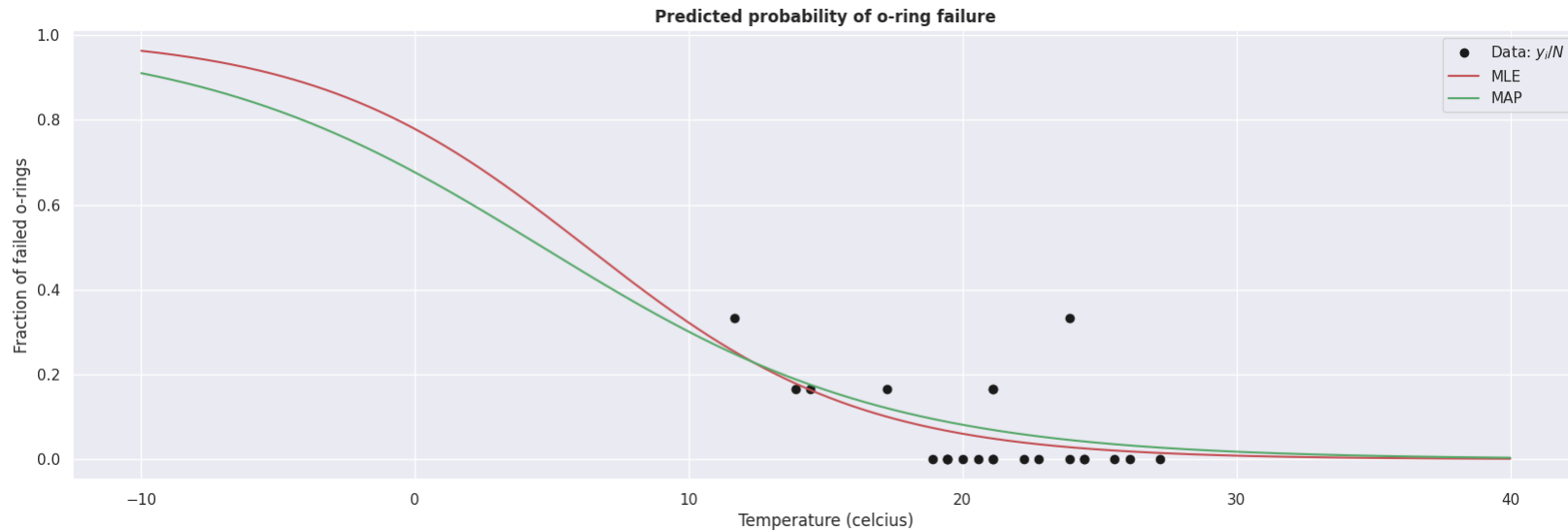
Solution

In [8]: # estimated functions for theta
theta_hat_MLE = lambda x: model.theta(x, alpha_MLE, beta_MLE)
theta_hat_MAP = lambda x: model.theta(x, alpha_MAP, beta_MAP)

# plot
fig, ax = plt.subplots(1, 1, figsize=(20, 6))
plot_data(ax, counts=False)
```

```
ax.plot(temp_pred, theta_hat_MLE(x_pred), 'r-', label='MLE')
ax.plot(temp_pred, theta_hat_MAP(x_pred), 'g-', label='MAP')
ax.legend()
ax.set_title('Predicted probability of o-ring failure', fontweight='bold')
```

Out[8]: Text(0.5, 1.0, 'Predicted probability of o-ring failure')



End of solution

Part 4: Approximating the posterior using a grid approximation

Rather than using point estimates, we will now turn to full Bayesian inference to be able to properly reason above the uncertainty of both parameters and predictions. In contrast to the Beta-binomial model, the logistic regression model is **not a conjugate model**, which means we cannot compute the posterior distribution $p(\alpha, \beta | \mathbf{y}, \mathbf{x})$ analytically. Instead, we will use a very simple **grid approximation** $q(\alpha, \beta)$ to represent the posterior distribution:

$$p(\alpha, \beta | \mathbf{y}, \mathbf{x}) \approx q(\alpha, \beta).$$

To construct the grid approximation $q(\alpha, \beta)$, we need three steps:

1. **Define grid:** First we need to define the grid for (α, β) . Typically, we choose an appropriate discrete set of values for α and β individually, i.e. $\{\alpha_i\}_{i=1}^I$ and $\{\beta_j\}_{j=1}^J$, and then take the (α, β) -grid to be the Cartesian product.
2. **Evaluation:** Next, we need to evaluate the function $p(\alpha, \beta | \mathbf{y}, \mathbf{x})$ on each point in the (α, β) -grid. Importantly, here we only need to evaluate the posterior **up to a normalization constant**, and hence, **we can evaluate the joint distribution instead of the posterior**. Moreover, for numerical reasons, we often evaluate the **logarithm of the joint distribution**.
3. **Normalization:** To ensure that our approximation is a valid distribution, we need to make it sum to 1. To achieve this, we compute the sum of $p(\mathbf{y}, \alpha, \beta | \mathbf{x})$ across all grid points to approximate the normalization constant Z .

That is,

$$q(\alpha_i, \beta_j) = \frac{1}{Z} p(\mathbf{y}, \alpha_i, \beta_j | \mathbf{x}) = \frac{1}{Z} \tilde{\pi}_{ij} = \pi_{ij},$$

where we have defined $\log \tilde{\pi}_{ij} = \log p(\mathbf{y}, \alpha_i, \beta_j | \mathbf{x})$ and $\pi_{ij} = \frac{1}{Z} \tilde{\pi}_{ij}$ for each point on the grid (α_i, β_j) . The normalization constant then becomes

$$Z = \sum_{i,j} p(\mathbf{y}, \alpha_i, \beta_j | \mathbf{x}) = \sum_{i,j} \tilde{\pi}_{ij} = \sum_{i=1}^I \sum_{j=1}^J \tilde{\pi}_{ij},$$

where the sum is over all the points in the grid. Note that $q(\alpha, \beta) = 0$ when evaluated on any point not belonging to the pre-defined (α, β) -grid.

As we will see later, the grid approximation makes it easy for us to

1. compute expectations with respect to the posterior
2. generate samples from the posterior, which will allow us to make inferences about the parameters α and β as well as make predictions while taking the uncertainty into account.

The purpose of the class `GridApproximation2D` given below is to approximate the posterior distribution of the logistic regression model using a grid approximation. We will re-use the code from the `Grid2D`-class to implemented step 1) and step 2) above. Hence, we basically also need to carry out the step 3.

Task 4.1: Complete the implementation of the `prep_approximation`-function below. Your task is to make sure that the `probability_grid`-variable contains all the π_{ij} defined above with the proper normalization.

```
In [9]: class GridApproximation2D(Grid2D):

    def __init__(self, alphas, betas, log_joint, threshold=1e-8, name="GridApproximation2D"):
        Grid2D.__init__(self, alphas, betas, log_joint, name)
        self.threshold = threshold
        self.prep_approximation()
        self.compute_marginals()
        self.sanity_check()

    def prep_approximation(self):

        # [num_alpha, num_beta]-sized matrix of the log joint evaluated on the grid
        self.log_joint_grid = self.values
        self.log_joint_grid = self.log_joint_grid - jnp.max(self.log_joint_grid)

        # evaluate joint for each point on the grid
        self.tilde_probabilities_grid = jnp.exp(self.log_joint_grid)

        # compute normalization constant
        self.Z = jnp.sum(self.tilde_probabilities_grid)

        # [num_alpha, num_beta]-matrix of \pi_{ij}-values summing to 1.
        self.probabilities_grid = self.tilde_probabilities_grid/self.Z

        # flatten for later convenience
        self.alphas_flat = self.alpha_grid.flatten()           # shape: [num_alpha*num_beta] = [num_outcomes]
        self.betas_flat = self.beta_grid.flatten()             # shape: [num_alpha*num_beta] = [num_outcomes]
        self.num_outcomes = len(self.alphas_flat)               # shape: scalar
        self.probabilities_flat = self.probabilities_grid.flatten() # [num_outcomes]

    def compute_marginals(self):
        self.pi_alpha = self.probabilities_grid.sum(1)
        self.pi_beta = self.probabilities_grid.sum(0)

        # compute marginal distribution using sum rule
    def compute_expectation(self, f):
        """ computes expectation of f(alpha, beta) wrt. the grid approximation """
        return jnp.sum(f(self.alphas_flat, self.betas_flat)*self.probabilities_flat, axis=0)

    def sample(self, key, num_samples=1):
        """ generate num_samples from the grid approximation distribution """
        idx = random.choice(key, jnp.arange(self.num_outcomes), p=self.probabilities_flat, shape=(num_samples, 1))
        return self.alphas_flat[idx], self.betas_flat[idx]

    def visualize(self, ax, scaling=8000, title='Grid approximation'):
        idx = self.probabilities_flat > self.threshold
        ax.scatter(self.alphas_flat[idx], self.betas_flat[idx], scaling*self.probabilities_flat[idx], label='$\pi_{ij}$')
        ax.set(xlabel='$\alpha$', ylabel='$\beta$')
        ax.set_title(title, fontweight='bold')

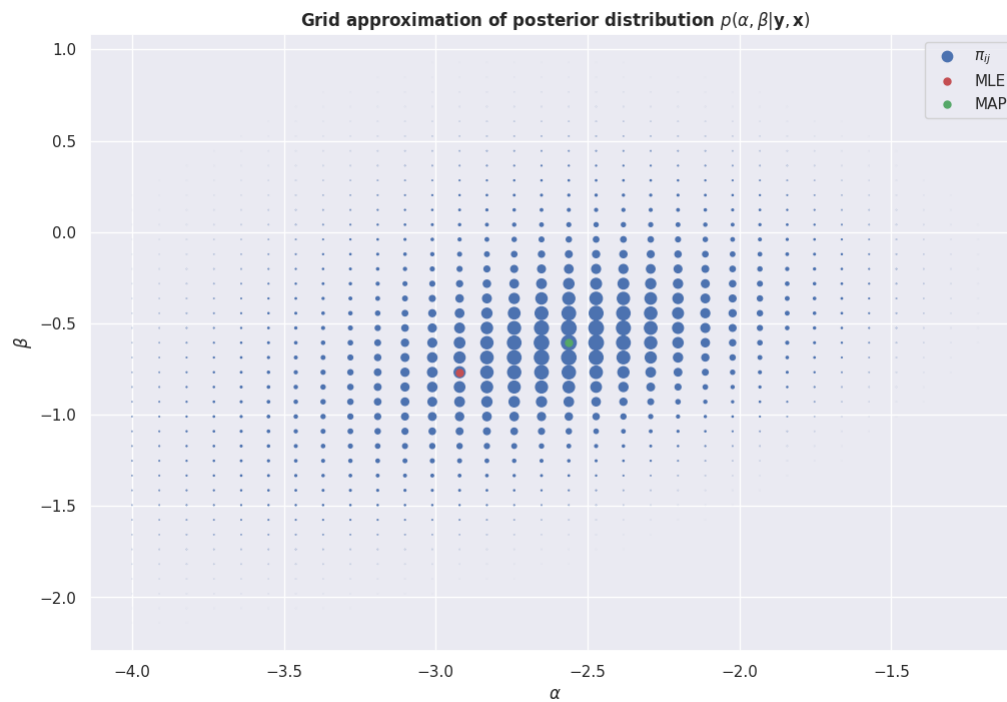
    def sanity_check(self):
        assert self.probabilities_grid.shape == self.grid_size, "Probability grid does not have shape [num_alphas, num_betas] (self.grid_size). Check your implementation."
        assert jnp.all(self.probabilities_grid >= 0), "Not all values in probability grid are non-negative. Check your implementation."
        assert jnp.allclose(self.probabilities_grid.sum(), 1), "Values in probability grid do not sum to one. Check your implementation."

# grid
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha)
betas = jnp.linspace(-4, 4, num_beta)

# model
model = LogisticRegression(x, y, N, sigma2_alpha=1.0, sigma2_beta=1.0)

# construct grid approximation for posterior
post_approx = GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')

# plot
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
post_approx.visualize(ax, title='Grid approximation of posterior distribution $p(\alpha, \beta) \propto \mathbf{y}, \mathbf{x}$')
ax.plot(alpha_MLE, beta_MLE, 'r.', label='MLE', markersize=10);
ax.plot(alpha_MAP, beta_MAP, 'g.', label='MAP', markersize=10);
ax.legend();
```



If the previous task was implemented correctly, the code above should produce a visualization of the grid approximation. The blue dots show the position of all the grid points, and the radius of the blue dots is controlled by the size of the corresponding weight π_{ij} . For easier interpretation, we do not plot points where π_{ij} is smaller than 10^{-8} .

Our first order of business after having obtained the posterior distribution is to produce estimates of the parameters α and β as well to quantify the uncertainty, i.e. we will estimate the posterior mean, posterior standard deviation and 95% credibility interval for both α and β . For this purpose, it is more convenient to work with the marginal posterior distributions of α and β , which can be readily obtained from the joint posterior approximation $q(\alpha, \beta)$ using the **sum rule**:

$$q(\alpha_i) = \sum_j q(\alpha_i, \beta_j) = \sum_j \pi_{ij} = \pi_i^\alpha$$

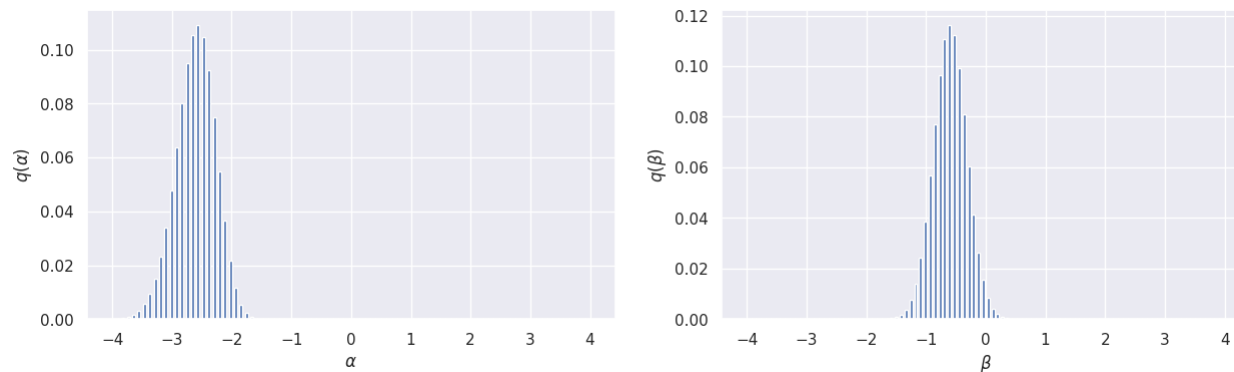
$$q(\beta_j) = \sum_i q(\alpha_i, \beta_j) = \sum_i \pi_{ij} = \pi_j^\beta,$$

i.e. $\pi_i^\alpha = q(\alpha_i)$ is the (approximate) marginal posterior probability of $p(\alpha = \alpha_i | \mathbf{y}, \mathbf{x})$.

Task 4.2: Go back up the class `GridApproximation2D` and implemented the `compute_marginals`-function. Use it to compute and plot the probability mass functions for the approximate posterior marginal distributions $q(\alpha)$ and $q(\beta)$ as a function of α and β , respectively.

Solution

```
In [ ]: # plot
fig, ax = plt.subplots(1, 2, figsize=(15, 4))
ax[0].bar(post_approx.alphas, post_approx.pi_alpha, width=0.05)
ax[0].set(xlabel='$\alpha$', ylabel='$q(\alpha)$')
ax[1].bar(post_approx.betas, post_approx.pi_beta, width=0.05)
ax[1].set(xlabel='$\beta$', ylabel='$q(\beta)$');
```

**End of solution**

Task 4.3: Compute the posterior mean and standard deviation for α and β using the grid approximation by completing the implementation of `DiscreteDistribution1D` -class below. Use the figures from the previous task to verify that your results are (approximately) correct.

Hints:

- The mean of a discrete random variable X with PMF $P(X = x)$ is given by $\mathbb{E}[X] = \sum_i x_i P(X = x_i)$, where the sum is over all the possible outcomes of X .
- The variance of a discrete random variable X with PMF $P(X = x)$ is given by $\mathbb{V}[X] = \sum_i (x_i - \mathbb{E}[X])^2 P(X = x_i)$, where the sum is over all the possible outcomes of X .

```
In [ ]: class DiscreteDistribution1D(object):

    def __init__(self, outcomes, probabilities, name='DiscreteDistribution'):
        """ represents discrete random variable X in terms of outcomes and probabilities """
        self.outcomes = outcomes
        self.probabilities = probabilities
        assert self.outcomes.shape == self.probabilities.shape
        self.name = name

    def CDF(self, x):
        """ P[X <= x] """
        idx = self.outcomes <= x
        return jnp.sum(self.probabilities[idx])

    def quantile(self, p):
        """ Q(p) = inf {x | p < CDF(x)} """
        cdf_values = jnp.cumsum(self.probabilities)
        idx = jnp.where(jnp.logical_or(p < cdf_values, jnp.isclose(p, cdf_values)))[0]
        return jnp.min(self.outcomes[idx])

    @property
    def mean(self):
        """ return scalar corresponding to the mean of the discrete distribution """
        return jnp.sum(self.outcomes * self.probabilities)

    @property
    def variance(self):
        """ return scalar corresponding to the variance of the discrete distribution """
        return jnp.sum((self.outcomes - self.mean)**2 * self.probabilities)

    def central_interval(self, interval_size=95):
        """ return tuple (lower, upper) corresponding to the central interval of the discrete distribution """
        c = 1.-interval_size/100.
        lower = self.quantile(c/2)
        upper = self.quantile(1-c/2)
        return jnp.array([lower, upper])

    def print_summary(self):
        print(f'Summary for {self.name}')
        print(f'\tMean:\t\t\t\t{self.mean:3.2f}')
        print(f'\tStd. dev.\t\t\t\t{jnp.sqrt(self.variance):3.2f}')
        print(f'\t95%-credibility interval:\t\t{self.central_interval()[0]:3.2f}, {self.central_interval()[1]:3.2f}\n')

# example use and sanity checks using classic six-sided die
outcomes, probs = jnp.arange(1,6+1), jnp.ones(6)/6
sixsided_die = DiscreteDistribution1D(outcomes, probs)
assert jnp.allclose(sixsided_die.mean, 3.5), f"The mean of a sixsided die should be 3.5, but the value was {sixsided_die.mean:3.2f}. Check your implementation."
```

```

assert jnp.allclose(sixsided_die.variance, 35/12), f"The variance of a sixsided die should be approximately 2.92, but the value was {sixsided_die.variance:3.2f}. Check your implementation."
assert jnp.allclose(sixsided_die.central_interval(95), jnp.array([1, 6])), f"The (approximate) 95% central interval of a sixsided die should be (1,6), but the value was {sixsided_die.central_interval(95)}. Check your implementation."

# summarize posterior marginals for alpha and beta
post_alpha_marginal = DiscreteDistribution1D(post_approx.alphas, post_approx.pi_alpha, "posterior distribution of alpha")
post_beta_marginal = DiscreteDistribution1D(post_approx.betas, post_approx.pi_beta, "posterior distribution of beta")

post_alpha_marginal.print_summary()
post_beta_marginal.print_summary()

Summary for posterior distribution of alpha
Mean: -2.61
Std. dev.: 0.33
95%-credibility interval: [-3.28, -2.02]

Summary for posterior distribution of beta
Mean: -0.60
Std. dev.: 0.28
95%-credibility interval: [-1.17, -0.04]

```

Compare the summary statistics to the plots above. Verify that your results are consistent before moving on.

Task 4.4: Compute the prior and posterior probability that $\beta < 0$

Solution

```

In [12]: prior_approx = GridApproximation2D(alphas, betas, model.log_prior, name='Prior')
prior_beta_marginal = DiscreteDistribution1D(prior_approx.betas, prior_approx.pi_beta, "prior distribution of beta")
print(f'p(beta < 0) = {prior_beta_marginal.CDF(0):3.2f}')
print(f'p(beta < 0|y) = {post_beta_marginal.CDF(0):3.2f}')

p(beta < 0) = 0.50
p(beta < 0|y) = 0.98

```

End of solution

Part 5: Propagating uncertainty from parameters to predictions

Using the grid approximation, we got a handle on the posterior distribution of α and β given the observed data, i.e. $p(\alpha, \beta | \mathbf{x}, \mathbf{y})$. We will now turn our attention to making predictions. For example, we want to predict the following quantities for a given temperature $t^* \in \mathbb{R}$:

- The probability of failure for each o-ring
- The probability of at least one o-ring failing
- The expected number of failing o-rings
- A 95%-credibility interval for then number of failed o-rings

Assuming we know the value of α and β , all quantities above are easily computed from the predictive likelihood

$$p(y^* | x^*, \alpha, \beta) = \text{Bin}(y^* | N^*, \alpha, \beta) \quad (12)$$

However, in practice we do not now the value of α and β , and therefore, we need to estimate them. The simplest approach for making predictions is to use a **plugin approximation** like we did in part 3. That is, we simply plug in one of our point estimates for α and β into the equation above, e.g. the MLE or MAP estimator. This is usually fast and easy to do, but the significant drawback is that it tends to produce **overconfident** predictions because it ignores the uncertainty for θ^* . Instead, we can propagate the uncertainty from the posterior distribution $q(\alpha, \beta)$ to the predictive distribution using **marginalization** via the sum:

$$p(y^* | \mathbf{y}, \mathbf{x}, x^*) = \int \int p(y^* | x^*, \alpha, \beta) p(\alpha, \beta | \mathbf{y}, \mathbf{x}) d\alpha d\beta = \mathbb{E}_{p(\alpha, \beta | \mathbf{y}, \mathbf{x})} [p(y^* | x^*, \alpha, \beta)] \approx \mathbb{E}_{q(\alpha, \beta)} [p(y^* | x^*, \alpha, \beta)]. \quad (13)$$

That is, when we make predictions, we average over the posterior distribution. **Instead of picking a single set of parameters, we consider all possible values for α and β and weigh them according to their posterior probability.** This may be abstract, but the grid approximation can help understand this more intuitively:

$$p(y^* | \mathbf{y}, \mathbf{x}, x^*) = \mathbb{E}_{p(\alpha, \beta | \mathbf{y})} [p(y^* | x^*, \alpha, \beta)] \approx \mathbb{E}_{q(\alpha, \beta)} [p(y^* | x^*, \alpha, \beta)] = \sum_{ij} p(y^* | x^*, \alpha_i, \beta_j) \pi_{ij}, \quad (14)$$

where we literally evaluate $p(y^* | x^*, \alpha, \beta)$ for each possible set of parameters (α_i, β_j) and weigh them by the posterior probability $\pi_{ij} = q(\alpha_i, \beta_j)$. We refer to the distribution $p(y^* | \mathbf{y}, \mathbf{x}, x^*)$ as the **posterior predictive distribution**. Recall, we can also cast the plugin approximations as approximation over the posterior predictive distribution, where the posterior is approximated by a posterior distribution that as all the mass on the single point $(\hat{\alpha}_{MLE}, \hat{\beta}_{MLE})$.

The code below uses the plugin approximation for the MLE/MAP estimator to compute and plot the posterior predictive distribution $p(y^* = k | \mathbf{y}, \mathbf{x}, x^*)$ for $k \in \{0, 1, \dots, 6\}$. The code also reports the mean and a 95% interval for each methods for each temperatures (assuming the tasks above have been solved).

Task 5.1: Compute and plot the full posterior predictive distribution $p(y^* = k | \mathbf{y}, \mathbf{x}, x^*)$ with $k \in \{0, 1, \dots, 6\}$ for $t = 25^\circ C$ and $t = 0^\circ C$.

```

In [13]: # target temperatures t^*
temp_stars = [25, 0]

# outcome space for y^*

```

```

ks = jnp.arange(0, 6+1)

# predictive likelihood  $p(y^{*}=k|x^{*}, \alpha, \beta)$ 
pred_lik = lambda k, x_star, alpha, beta: binom_dist.pmf(k=k, n=6, p=model.theta(x_star, alpha, beta))

# prep figure
fig, ax = plt.subplots(2, 3, figsize=(20, 10))

mean_and_interval_label = lambda dist: f'Mean: {dist.mean:3.2f}, 95%-interval: [{dist.central_interval()[0]:d}, {dist.central_interval()[1]:d}]'

for n, t_star in enumerate(temp_stars):

    # standardize to get  $x^{*}$ 
    x_star = standardize(t_star)

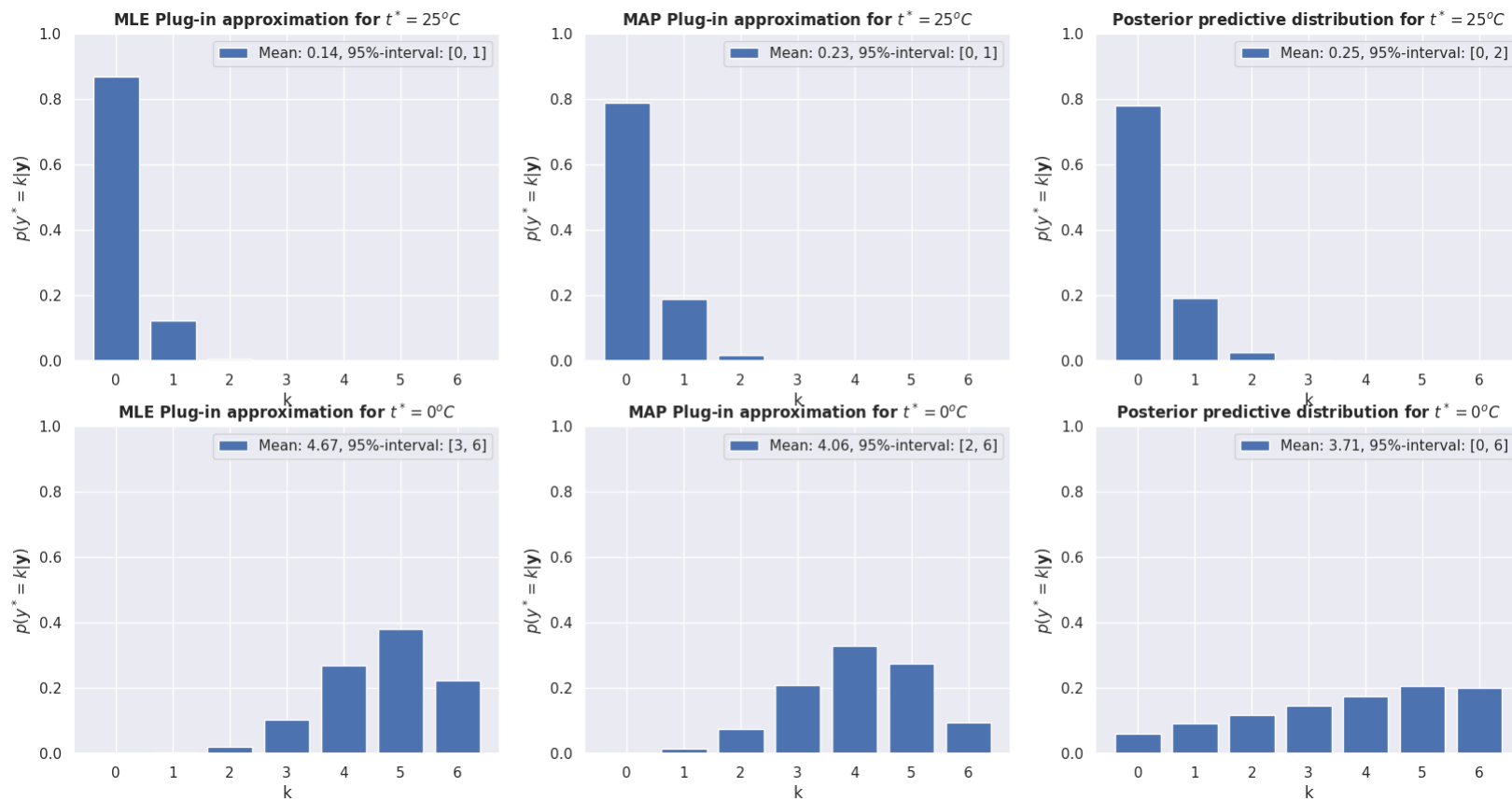
    # compute plugin approximations using MLE and MAP. Shape must be [7] (matching the shape of ks)
    pred_lik_probs_MLE = pred_lik(ks, x_star, alpha_MLE, beta_MLE)
    pred_lik_probs_MAP = pred_lik(ks, x_star, alpha_MAP, beta_MAP)

    # compute posterior predictive distribution as a weighted averaged over all parameter vales. Shape must be [7] (matching the shape of ks)
    pred_lik_bayes_probs = jnp.sum(pred_lik(ks, x_star, post_approx.alphas_flat[:, None], post_approx.betas_flat[:, None])*post_approx.proBABILITIES_flat[:, None], axis=0)

    # prepare for plotting etc
    pred_dist_MLE = DiscreteDistribution1D(ks, pred_lik_probs_MLE, name='MLE Plug-in approximation')
    pred_dist_MAP = DiscreteDistribution1D(ks, pred_lik_probs_MAP, name='MAP Plug-in approximation')
    pred_dist_bayes = DiscreteDistribution1D(ks, pred_lik_bayes_probs, name="Posterior predictive distribution")

    # plot
    for i, dist in enumerate([pred_dist_MLE, pred_dist_MAP, pred_dist_bayes]):
        ax[n, i].bar(ks, dist.proBABILITIES, label=mean_and_interval_label(dist))
        ax[n, i].set(xlabel='k', ylabel=f' $p(y^{*} = k | \mathbf{x}^{*})$ ', ylim=(0, 1))
        ax[n, i].set_ylim((0, 1))
        ax[n, i].set_title(f'{dist.name} for  $t^{*} = \{t\_star\}^{*}$ ', fontweight='bold')
        ax[n, i].set(xlabel='k', ylabel=f' $p(y^{*} = k | \mathbf{x}^{*})$ ', ylim=(0, 1))
        ax[n, i].legend()

```



Task 5.2: Comment on the differences between the results obtained using plugin approximations, i.e. the MLE/MAP estimators, and the posterior predictive distributions. How does the three approximations compare? In regions with data? In regions without data?

Note: This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.

In the very last of the exercise, we will visualize the posterior for $f(x)$ and $\theta(x)$ to build over intuition. We will start by drawing a set of random samples of (α, β) from the posterior approximation and then compute and visualize $f(x)$ and $\theta(x)$ for each sample. We will repeat this process for the prior distribution $p(\alpha, \beta)$.

Task 5.3: The code below generates `num_samples` of α, β from both the prior and posterior and plots them. Compute $f(x_{\text{pred}})$ and $\theta(x_{\text{pred}})$ for each of these samples and store them in the variables `f_samples` and `theta_samples` pre-defined below.

```
In [14]: # data for predictions
temp_pred = jnp.linspace(-10, 40, 500)
x_pred = standardize(temp_pred)
N_pred = 6

# number of samples plot
num_samples = 100

# grid
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha)
betas = jnp.linspace(-4, 4, num_beta)

# set up prior and posterior for visualization purposes
model = LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')
prior_approx = GridApproximation2D(alphas, betas, model.log_prior, name='Prior')
```



```

# specify seed
seed = 123
key = random.PRNGKey(seed)

# plot for each distribution...
fig, ax = plt.subplots(2, 3, figsize=(20, 8))
for idx_plot, dist in enumerate([prior_approx, post_approx]):
    key, subkey = random.split(key)

    # plot data and contours of distribution
    plot_data(ax[idx_plot, 2], counts=False)
    dist.plot_contours(ax[idx_plot, 0], f=jnp.exp, color=colors[idx_plot])

    # get (alpha, beta)-samples to visualize
    alpha_samples, beta_samples = dist.sample(subkey, num_samples)

    # compute samples of f(x_pred) and theta(x_pred). Shape must be [num_samples, len(temp_pred)] for both f_samples and theta_samples
    f_samples = model.f(x_pred, alpha_samples, beta_samples)
    theta_samples = model.theta(x_pred, alpha_samples, beta_samples)

    # plot
    ax[idx_plot, 0].plot(alpha_samples, beta_samples, '.', color=colors[idx_plot])

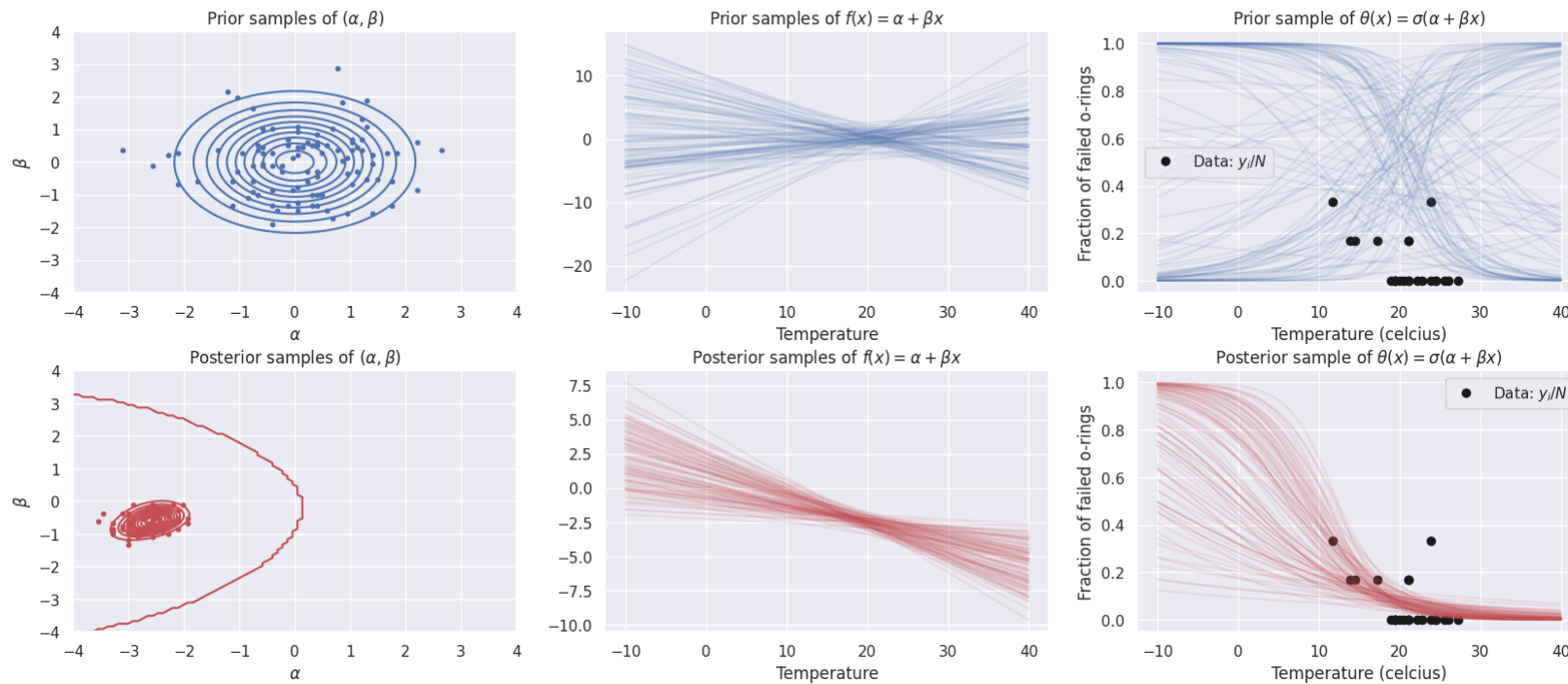
    # plot each sample of f and theta
    ax[idx_plot, 1].plot(temp_pred, f_samples.T, '', color=colors[idx_plot], alpha=0.1)
    ax[idx_plot, 2].plot(temp_pred, theta_samples.T, '', color=colors[idx_plot], alpha=0.1)
    ax[idx_plot, 2].legend()

    # title and labels
    ax[idx_plot, 0].set(title=f'{dist.name} samples of $(\\alpha, \\beta)$')
    ax[idx_plot, 1].set(title=f'{dist.name} samples of $f(x) = \\alpha + \\beta x$, xlabel='Temperature')
    ax[idx_plot, 2].set(title=f'{dist.name} sample of $\\theta(x) = \\sigma(\\alpha + \\beta x)$')

fig.suptitle('Prior and posterior predictions', fontweight='bold')
fig.subplots_adjust(hspace=0.3)

```

Prior and posterior predictions



Task 5.4: Study the code and plots above and make sure you can explain the following:

- What is plotted in each of the six panels?
- What is the relationship between each of the dots in the left columns and the curves in the center and right columns?
- How the uncertainty on $f(x)$ and $\theta(x)$ is controlled by the distribution of (α, β) ?
- How well does each of the prior samples match the observed data?
- How well does each of the posterior samples match the observed data?

Note: This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.

We can also use posterior samples to summarize the pointwise posterior mean and 95% intervals of $f(x)$ and $\theta(x)$:

```
In [15]: # set seed
seed = 0
key = random.PRNGKey(seed)

# set up prior and posterior for visualization purposes
model = LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')
prior_approx = GridApproximation2D(alphas, betas, model.log_prior, name='Prior')

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 6))
plot_data(ax[0], counts=False)
plot_data(ax[1], counts=False)

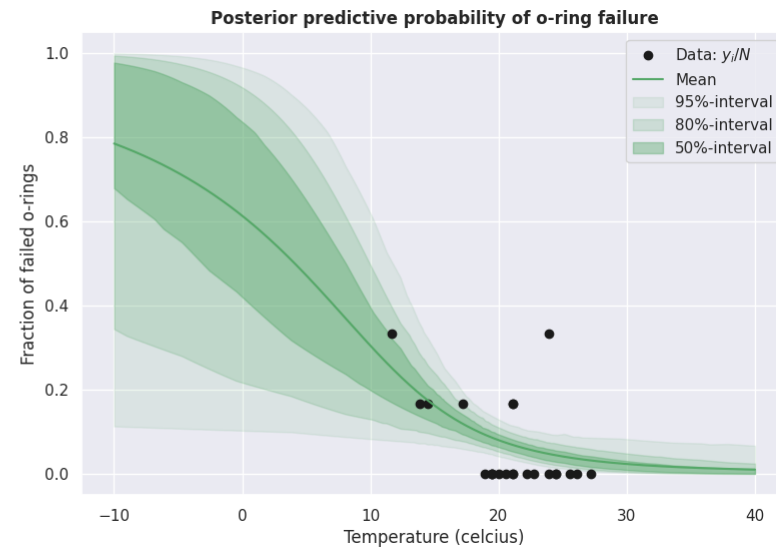
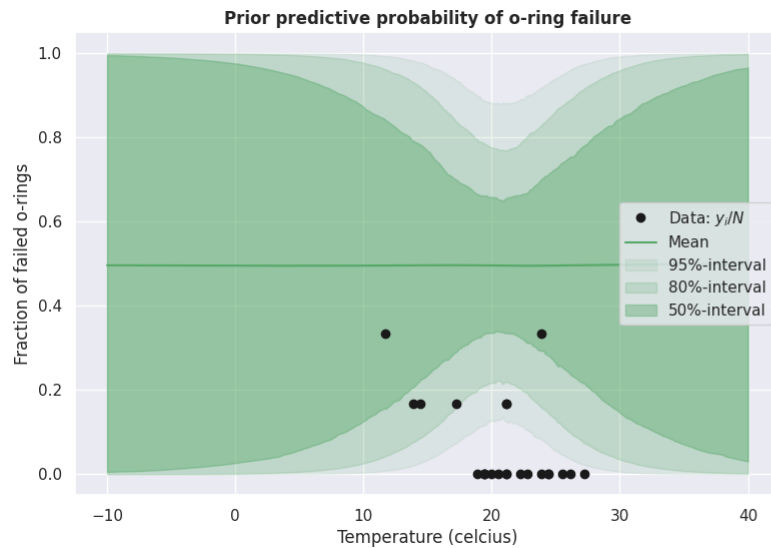
num_samples = 2000

for idx_plot, dist in enumerate([prior_approx, post_approx]):
    key, subkey = random.split(key)

    alpha_samples, beta_samples = dist.sample(subkey, num_samples)

    theta_samples = model.theta(x_pred, alpha_samples, beta_samples)
    theta_mean = theta_samples.mean(0)
    theta_lower = jnp.quantile(theta_samples, 0.025, axis=0)
    theta_upper = jnp.quantile(theta_samples, 1-0.025, axis=0)

    ax[idx_plot].plot(temp_pred, theta_mean, 'g-', label='Mean')
    for interval_size, b in [(0.95, 0.1), (0.8, 0.2), (0.5, 0.4)]:
        c = 1-interval_size
        theta_lower = jnp.quantile(theta_samples, c/2, axis=0)
        theta_upper = jnp.quantile(theta_samples, 1-c/2, axis=0)
        ax[idx_plot].fill_between(temp_pred, theta_lower, theta_upper, color='g', alpha=b, label=f'{interval_size*100:2.0f}%-interval')
    ax[idx_plot].legend()
    ax[idx_plot].set_title(f'{dist.name} predictive probability of o-ring failure', fontweight='bold')
```



Task 5.5: Compute and plot the posterior mean and a 95% credibility interval for the probability of obtaining at least one failure among the 6 o-rings

Solution

```
In [16]: # set seed and number of samples
seed = 0
key = random.PRNGKey(seed)
num_samples = 2000

# set up prior and posterior for visualization purposes
model = LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')

alpha_samples, beta_samples = post_approx.sample(key, num_samples)
theta_samples = model.theta(x_pred, alpha_samples, beta_samples)

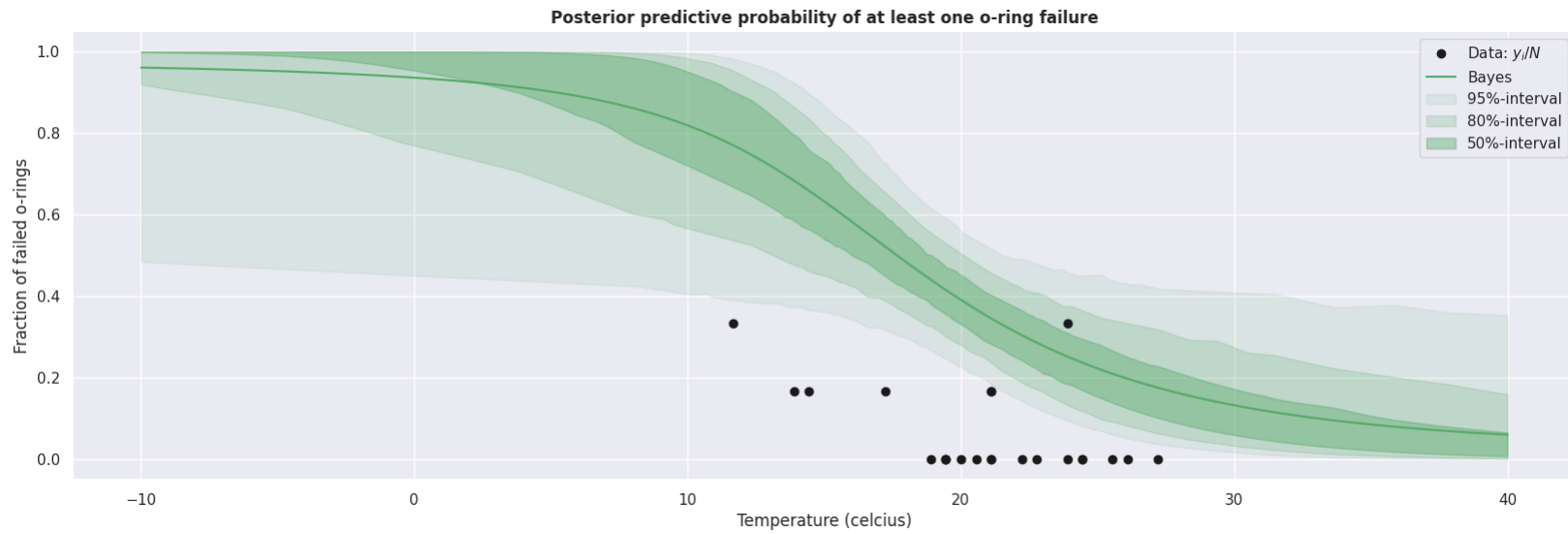
# probability of at least one failure wrt. a binomial post_approximation
prob_at_least_one_failure = 1 - (1-theta_samples)**N_pred

# compute mean and interval
post_mean = prob_at_least_one_failure.mean(0)
post_lower = jnp.quantile(prob_at_least_one_failure, 0.025, axis=0)
post_upper = jnp.quantile(prob_at_least_one_failure, 1-0.025, axis=0)

# plot
fig, ax = plt.subplots(1, 1, figsize=(20, 6))
plot_data(ax, counts=False)

ax.plot(temp_pred, post_mean, 'g-', label='Bayes')
for interval_size, b in [(0.95, 0.1), (0.8, 0.2), (0.5, 0.4)]:
    c = 1-interval_size
    post_lower = jnp.quantile(prob_at_least_one_failure, c/2, axis=0)
    post_upper = jnp.quantile(prob_at_least_one_failure, 1-c/2, axis=0)
    ax.fill_between(temp_pred, post_lower, post_upper, color='g', alpha=b, label=f'interval_size*100:2.0f%-interval')
ax.legend()
ax.set_title(f'{post_approx.name} predictive probability of at least one o-ring failure', fontweight='bold')

Out[16]: Text(0.5, 1.0, 'Posterior predictive probability of at least one o-ring failure')
```



End of solution